Compiler Project

Syntax Analysis

Roland Greffe & Pascal Fontaine

University of Liège

February 24, 2026

Outline

Outline

Syntax Analysis

Converts a stream of tokens into an **abstract syntax tree** (AST)

Detects **syntactically invalid** source code

`vsopc -p example.vsop` should:

- If the source file is syntactically valid, return 0 and print on `stdout` the abstract syntax tree following the format given in the statement
- Otherwise, return a non-zero value and print on `stderr` (at least) one syntax or lexical error

Assignment

Due the 18th of March

Automated tests worth 5% of your grade

You can use a **parser generator** (e.g. bison, PLY, ANTLR)

Support for **custom tests** in `tests` subfolder

Two modes: `-p` and `-l`

Input Format

```
class List {
    isNil() : bool { true }
    length() : int32 { 0 }
  }
class Cons extends List {
    head : int32;
    tail : List;
    init(hd : int32, tl : List) : Cons {
        head <- hd;
        tail <- tl;
        self
      }
    head() : int32 { head }
    isNil() : bool { false }
    length() : int32 { 1 + tail.length() }
  }
...
```

Output Format

```
[Class(List, Object, [],
       [Method(isNil, [], bool, true),
        Method(length, [], int32, 0)]),
 Class(Nil, List, [], []),
 Class(Cons, List, [Field(head, int32),
                    Field(tail, List)],
       [Method(init, [hd : int32, tl : List], Cons,
               [Assign(head, hd),
                Assign(tail, tl), self]),
        Method(head, [], int32, head),
        Method(isNil, [], bool, false),
        Method(length, [], int32,
               BinOp(+, 1, Call(tail, length, [])))]),
 ...
```

Error Management

Error messages on `stderr`, fail with code $\neq 0$

```
input_file.vsop:4:12: syntax error: description
```

Tests do not check the positions

Automated tests don't check the description, but **we do** !

Syntax error reporting is **challenging**, see lectures !

Lexical errors can still happen !

Conflicts

If your parser generator creates bottom-up parsers, they may face:

- **shift/reduce** conflicts
- **reduce/reduce** conflicts

Try to solve them, especially for the final deadline of the project (in May)

You will be penalized if you don't !

Questions and (Possibly) Answers

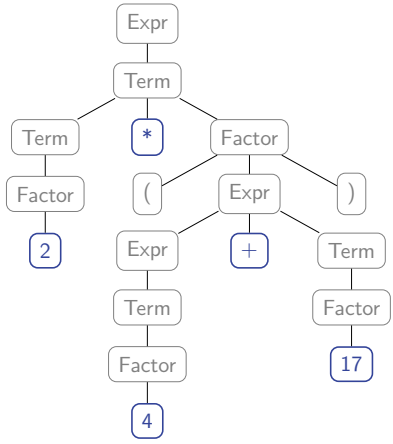
Outline

A Concrete Parse Tree is Very Redundant

A parse tree represents the **syntactic structure** of a string (which can be a mathematical expression, a source code, etc)

But it is very **redundant**

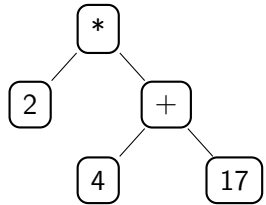
E.g., for `2 * (4 + 17)`:

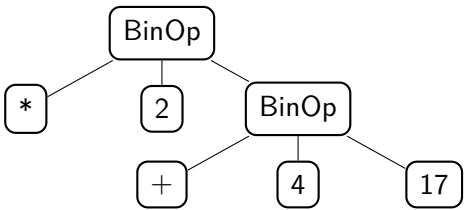


Use an Abstract Syntax Tree (AST)

An abstract parse tree **does not represent** all the details, but keep the important ones

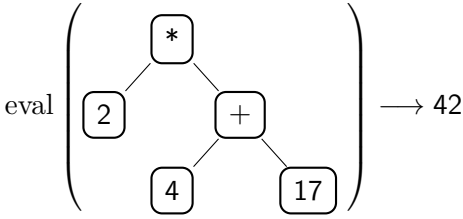
E.g., for `2 * (4 + 17)`:





Processing Abstract Syntax Trees

Example: evaluating arithmetic expressions



In your compiler:

- Generate the tree (**syntax analysis**) and print it (AST dump)
- Annotate it with types (**semantic analysis**)
- Generate code from it (**code generation**)

Outline

Avoid Generic Trees

```
class ASTNode {
    private List<ASTNode> children;
    private String value;
    ...
    public double eval() {
        if (children.length() == 3) {
            if (children.get(1).value == "+") {
                return children.get(0).eval()
                    + children.get(2).eval();
            } else if ...
```

Seems economical, but **hard to read** and **error-prone**

OO Inheritance-based Approach

```
abstract class Expr { public abstract double eval(); }
class Add extends Expr {
    Expr lhs;
    Expr rhs;
    ...
    public double eval() {
        return lhs.eval() + rhs.eval();
    }
}
class Sub extends Expr {
```

Simple, good encapsulation, easy to add new nodes

Logic of a pass spread across many classes, **Inheritance has a cost**

Approach with Introspection is Generally Slow

```
double eval(Expr e) {
    if (e instanceof Num) {
        return ((Num) e).getValue();
    } else if (e instanceof Add) {
        Add add = (Add) e;
        return eval(add.getLeft()) + eval(add.getRight());
    } else if (e instanceof Sub) {
        ...
```

Java's `instanceof` operator is expensive

Long chain of `if-else if` $\implies \mathcal{O}(n)$ checks per node!

Except in selected languages (e.g. Darts)

The Visitor Design Pattern

Idea: **Separate** the code that **stores the data** from the code that **manipulates it**

- **Data classes** store the data and implements the `accept` methods that allow a visitor to manipulate their data
- **Visitor classes** perform operations on the data classes: they implement a `visit` method for each type of data class

The Visitor Design Pattern: Interface

```
interface Visitor<R> {
    public R visit(Num num);
    public R visit(Add add);
    public R visit(Sub sub);
    ...
}
```

An `interface` with one `visit` method per AST node.

Parameterized over return type.

The Visitor Design Pattern: accept()

```
abstract class Expr {
    abstract public <R> R accept(Visitor<R> v);
}
class Num extends Expr {
    public <R> R accept(Visitor<R> v) {
        return v.visit(this);
    }
    ...
}
class Add extends Expr {
    public <R> R accept(Visitor<R> v) {
        return v.visit(this);
    }
    ...
}
```

The Visitor Design Pattern: Use

```
class EvalVisitor implements Visitor<Double> {
    public Double visit(Num num) {
        return num.getValue();
    }
    public Double visit(Add add) {
        return add.lhs.accept(this) + add.rhs.accept(this);
    }
    public Double visit(Sub sub) {
        return sub.lhs.accept(this) - sub.rhs.accept(this);
    }
    ...
}
```

The Visitor Design Pattern: Use

```
class EvalVisitor implements Visitor<Double> {
    public Double visit(Num num) {
        return num.getValue();
    }
    public Double visit(Add add) {
        return add.lhs.accept(this) + add.rhs.accept(this);
    }
    public Double visit(Sub sub) {
        return sub.lhs.accept(this) - sub.rhs.accept(this);
    }
    ...
}
```

Functional, type-safe, double dispatch faster than introspection

Little boilerplate, slightly heavy syntactically

Questions and (Possibly) Answers